



Explore | Expand | Enrich

<Codemithra />TM

MINIMUM STACK





MINIMUM STACK

EXPLANATION

- A stack is a type of data structure where items are added and removed from the top of the stack according to the Last-In-First-Out (LIFO) principle.
- Finding the smallest element poses challenges as the stack does not naturally allow direct access to it, but gaining access to and deleting elements from the top of the stack is simple.

MINIMUM STACK

OPERATIONS

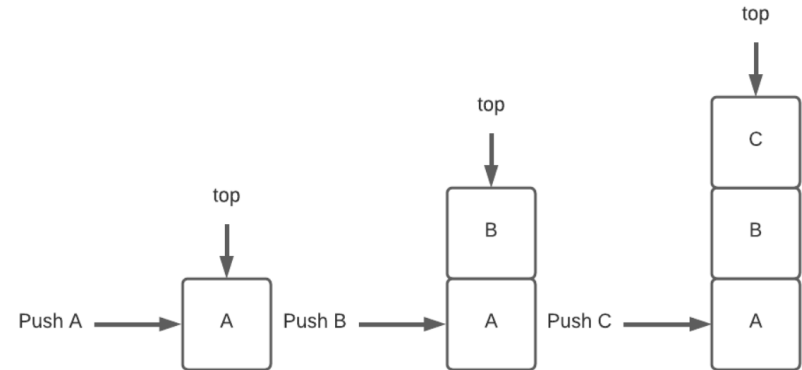
1. Push
2. Pop
3. Top
4. Getmin

MINIMUM STACK

1. PUSH

Push an element **into** the stack **by performing this action**.

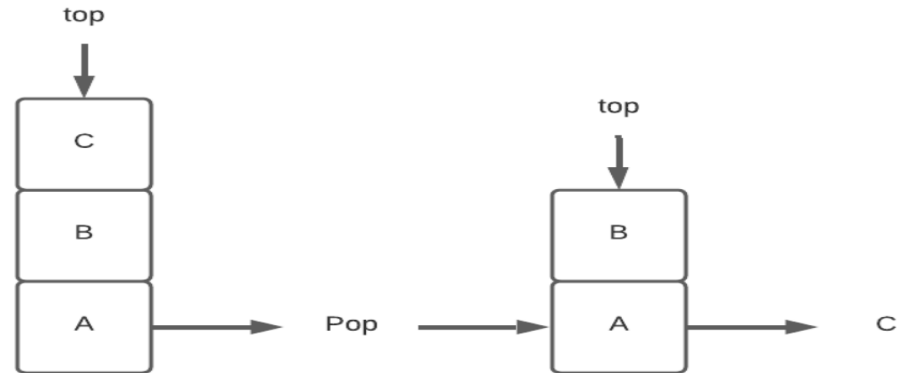
The element is **added**, or "pushed," to the top of the stack, **and the size of the stack is increased by one as a result**.



MINIMUM STACK

2. POP

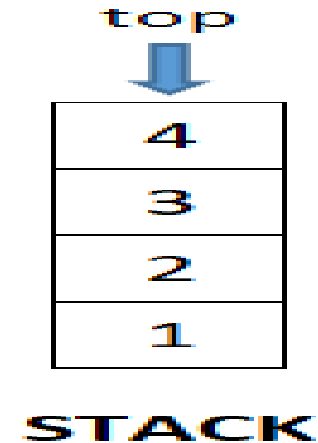
Stack operation performed to remove an element from the stack. The element is removed, or popped from the top



MINIMUM STACK

3.TOP

Stack operation to get the element stored at the top of the stack. The element is not removed using this stack operation.



MINIMUM STACK

4.GETMIN()

Retrieve the minimum element in the stack.

Stack



Stack with min



MINIMUM STACK

ALGORITHM

1. Initialize two stacks: a main stack to store elements and a minimum stack to track the minimum elements.
2. During the push operation:

Push the new element onto the main stack.

3. Compare the new element with the top element of the minimum stack:
4. If the new element is smaller or equal, push it onto the minimum stack.
5. Otherwise, push the current top element of the minimum stack onto itself.

MINIMUM STACK

ALGORITHM

5. During the pop operation:

Pop the top element from both the main stack and the minimum stack.

6. To retrieve the minimum element, simply peek at the top of the minimum stack.

MINIMUM STACK

PSEUDOCODE

Initialize mainStack and minStack

Push(element):

 Push element onto mainStack

 If minStack is empty OR $\text{element} \leq \text{top}(\text{minStack})$:

 Push element onto minStack

Pop():

 If $\text{top}(\text{mainStack}) == \text{top}(\text{minStack})$:

 Pop from minStack

 Pop from mainStack

GetMin():

 Return $\text{top}(\text{minStack})$

MINIMUM STACK

SAMPLE PROGRAM 1

INPUT

```
stack.push(5);  
stack.push(2);  
stack.push(7);  
stack.push(1);  
int minElement = stack.getMin();  
stack.pop();  
int topElement = stack.top();  
int newMinElement = stack.getMin();
```

MINIMUM STACK

SAMPLE PROGRAM 1

OUTPUT

Minimum Element: 1

Top Element: 7

After Pop New Minimum Element: 2

MINIMUM STACK

```
import java.util.Stack;
class MinStack {
    Stack<Integer> stack; // Stack to store
elements
    Stack<Integer> minStack; // Stack to track
minimum elements
    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }
    public void push(int element) {
        stack.push(element); // Push the element
onto the main stack
        // Update the minimum stack
        if (minStack.isEmpty() || element <=
minStack.peek()) {
            minStack.push(element); // Push the
element onto the minimum stack
        }
    }
}
```

```
import java.util.Stack;
class MinStack {
    Stack<Integer> stack; // Stack to store
elements
    Stack<Integer> minStack; // Stack to track
minimum elements
    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }
    public void push(int element) {
        stack.push(element); // Push the element
onto the main stack
        // Update the minimum stack
        if (minStack.isEmpty() || element <=
minStack.peek()) {
            minStack.push(element); // Push the
element onto the minimum stack
        }
    }
}
```

MINIMUM STACK

```
public int getMin() {
    if (!minStack.isEmpty()) {
        return minStack.peek(); // Return
the minimum element from the minimum stack
    }
    // Stack is empty, return -1 or throw an
exception as desired
    return -1;
}
public static void main(String[] args) {
    MinStack stack = new MinStack();
    // Push some elements onto the stack
    stack.push(5);
    stack.push(2);
    stack.push(7);
    stack.push(1);
```

```
// Retrieve and print the minimum element
    int minElement = stack.getMin();
    System.out.println("Minimum Element: " +
minElement); // Output: Minimum Element: 1
    // Pop an element from the stack
    stack.pop();
    // Retrieve and print the top element
    int topElement = stack.top();
    System.out.println("Top Element: " +
topElement); // Output: Top Element: 7
    // After pop
    int newMinElement = stack.getMin();
    System.out.println("After Pop New
Minimum Element: " + newMinElement);
    }
}
```



MINIMUM STACK

SAMPLE PROGRAM 1

Input

```
["push", "push", "pop", "push", "push", "peek", "push", "getMin", "pop", "getMin"]  
[ [2 ], [3], [ ], [4], [ -1], [ ], [6], [ ], [ ], [ ] ]
```

Output

```
[-1]
```

MINIMUM STACK

SAMPLE PROGRAM 1

Explanation

```
MinStack minStack = new MinStack();  
minStack.push(2);  
minStack.push(3);  
minStack.pop();  
minStack.push(4);  
minStack.push(-1);  
minStack.peek();  
minStack.push(6);  
minStack.getMin(); // return -1  
minStack.pop();  
minStack.getMin(); // return -1
```



MINIMUM STACK

```
import java.util.Stack;
//class
class MinStack {
private Stack<Integer> stack;
private Stack<Integer> minStack;
//constructor -memory creation
public MinStack() {
stack = new Stack<>();
minStack = new Stack<>();
}
public void push(int val) {
stack.push(val);
if (minStack.isEmpty() || val <=
minStack.peek())
{
//or operation for check any one is true
minStack.push(val);
}
}
```

```
public void pop() {
    if (!stack.isEmpty()) {
        if
(stack.peek().equals(minStack.peek())) {
            minStack.pop();
        }
        stack.pop();
    }
}
public int top() {
    return stack.peek();
}
public int getMin() {
    return minStack.peek();
}
}
```

MINIMUM STACK

```
public class Main {  
    public static void main(String[] args) {  
        MinStack minStack = new MinStack();  
        minStack.push(2);  
        minStack.push(3);  
        minStack.pop();  
        minStack.push(4);  
        minStack.push(-1);  
        minStack.top();  
        minStack.push(6);  
        minStack.getMin();  
        minStack.pop();  
        minStack.getMin();  
        System.out.println("Current Min: " + minStack.getMin());  
    }  
}
```

MINIMUM STACK

TIME COMPLEXITY AND SPACE COMPLEXITY

The time and space complexity of this code is $O(1)$ for each of the push, pop, top, and getMin operations because the code's execution time and memory usage do not depend on the number of elements in the stack.

MINIMUM STACK

```
public class Main {  
    public static void main(String[] args) {  
        LinkedList list = new LinkedList();  
        list.append(1);  
        list.append(2);  
        list.append(3);  
        list.append(4);  
        list.append(5);  
  
        System.out.println("Original list:");  
        list.printList();  
  
        list.segregateEvenOdd();  
  
        System.out.println("Segregated list (even before odd):");  
        list.printList();  
    }  
}
```

MINIMUM STACK

Time complexity and space complexity

Time complexity: $O(N)$, where N is the number of nodes in the linked list.

Space complexity: $O(1)$, constant space is used for the pointers and temporary variables.

INTERVIEW QUESTIONS

1. Why is it necessary to use an additional stack for retrieving the minimum element?

Answer: The additional stack, known as the min stack, is used to keep track of the minimum element at each point in the main stack. It ensures constant-time retrieval of the minimum element without having to traverse the entire stack every time. This approach improves efficiency and maintains the LIFO property of the stack.

INTERVIEW QUESTIONS

2. What is the time complexity of retrieving the minimum element from the stack using the Min Stack approach?

Answer: The time complexity to retrieve the minimum element using the Min Stack approach is constant, $O(1)$. Since the minimum element is always stored at the top of the min stack, accessing it does not depend on the size of the main stack.

INTERVIEW QUESTIONS

3. Does using the Min Stack approach increase the space complexity?

Answer: Yes, using the Min Stack approach increases the space complexity. In addition to the main stack, an extra stack (the min stack) is required to store the minimum elements. However, the space complexity remains proportional to the number of elements in the stack.

INTERVIEW QUESTIONS

4. How does the Min Stack approach handle scenarios when the minimum element is popped from the main stack?

Answer: When the minimum element is popped from the main stack, the Min Stack approach ensures consistency by also popping the corresponding minimum element from the min stack. This guarantees that the top of the min stack always represents the minimum element present in the main stack at any given time.

INTERVIEW QUESTIONS

5. Can the Min Stack approach be used with any type of stack implementation?

Answer: Yes, the Min Stack approach can be used with any type of stack implementation, whether it's implemented using an array or a linked list. The key idea is to maintain a separate stack for tracking the minimum elements alongside the main stack.



<https://learn.codemithra.com>



THANK YOU



+91 78150 95095



codemithra@ethnus.com



www.codemithra.com